

Human Activity Recognition Simulink Model for Smartphone Deployment

This example shows how to prepare a Simulink® model that classifies human activity based on smartphone sensor signals for code generation and smartphone deployment. The example provides a Simulink models that is ready for deployment to an Android® device. After you install the required support package, train the classification model and deploy the Simulink model to the device.

This example uses:

Statistics and Machine Learning Toolbox
Simulink
DSP System Toolbox
Signal Processing Toolbox
Simulink Support Package for Android Devices

 [Copy Command](#)

```
load humanactivity
```

 Get ▼

The `humanactivity` data set contains 24,075 observations of five different physical human activities: Sitting, Standing, Walking, Running, and Dancing. Each observation has 60 features extracted from acceleration data measured by smartphone accelerometer sensors. The data set contains the following variables:

- `actid` — Response vector containing the activity IDs in integers: 1, 2, 3, 4, and 5 representing Sitting, Standing, Walking, Running, and Dancing, respectively
- `actnames` — Activity names corresponding to the integer activity IDs
- `feat` — Feature matrix of 60 features for 24,075 observations
- `featlabels` — Labels of the 60 features

The Sensor HAR (human activity recognition) App [1] was used to create the `humanactivity` data set. When measuring the raw acceleration data with this app, a person placed a smartphone in a pocket so that the smartphone was upside down and the screen faced toward the person. The software then calibrated the measured raw data accordingly and extracted the 60 features from the calibrated data. For details about the calibration and feature extraction, see [2] and [3], respectively. The Simulink model described later also uses the raw acceleration data and includes blocks for calibration and feature extraction.

Prepare Data

This example uses 90% of the observations to train a model that classifies the five types of human activities and 10% of the observations to validate the trained model. Use `cvpartition` to specify a 10% holdout for the test set.

```
rng('default') % For reproducibility
Partition = cvpartition(actid,'Holdout',0.10);
trainingInds = training(Partition); % Indices for the training set
XTrain = feat(trainingInds,:);
YTrain = actid(trainingInds);
testInds = test(Partition); % Indices for the test set
XTest = feat(testInds,:);
YTest = actid(testInds);
```

 Get ▼

Convert the feature matrix `XTrain` and the response vector `YTrain` into a table to load the training data set in the Classification Learner app.

```
tTrain = array2table([XTrain YTrain]);
```

 Get ▼

Specify the variable name for each column of the table.

```
tTrain.Properties.VariableNames = [featlabels' 'Activities'];
```

 Get ▼

Train Boosted Tree Ensemble Using Classification Learner App

Train a classification model by using the Classification Learner app. To open the Classification Learner app, enter `classificationLearner` at the command line. Alternatively, click the **Apps** tab, and click the arrow at the right of the **Apps** section to open the gallery. Then, under **Machine Learning and Deep Learning**, click **Classification Learner**.

On the **Classification Learner** tab, in the **File** section, click **New Session** and select **From Workspace**.

In the New Session from Workspace dialog box, click the arrow for **Data Set Variable**, and then select the table `tTrain`. Classification Learner detects the predictors and the response from the table.

New Session from Workspace

Data set

Data Set Variable: tTrain 21668x61 table

Response:

☒ From data set variable

☐ From workspace

Activities: double

Predictors

	Name	Type	Range
☒	TotalAccXMean	double	-0.453039 .. 0.834224
☒	TotalAccYMean	double	-1.24491 .. 1.66989
☒	TotalAccZMean	double	-0.989318 .. 0.36021
☒	BodyAccXRMS	double	0.00136851 .. 0.99214
☒	BodyAccYRMS	double	0.00244462 .. 2.23241
☒	BodyAccZRMS	double	0.000647098 .. 1.42422

Add All Remove All

[How to prepare data](#) Refresh

Response variable is numeric. Distinct values will be interpreted as class labels.

Validation

Validation Scheme: Cross-Validation

Protects against overfitting. For data not set aside for testing, the app partitions the data into folds and estimates the accuracy on each fold.

Cross-validation folds: 5

[Read about validation](#)

Test

☐ Set aside a test data set

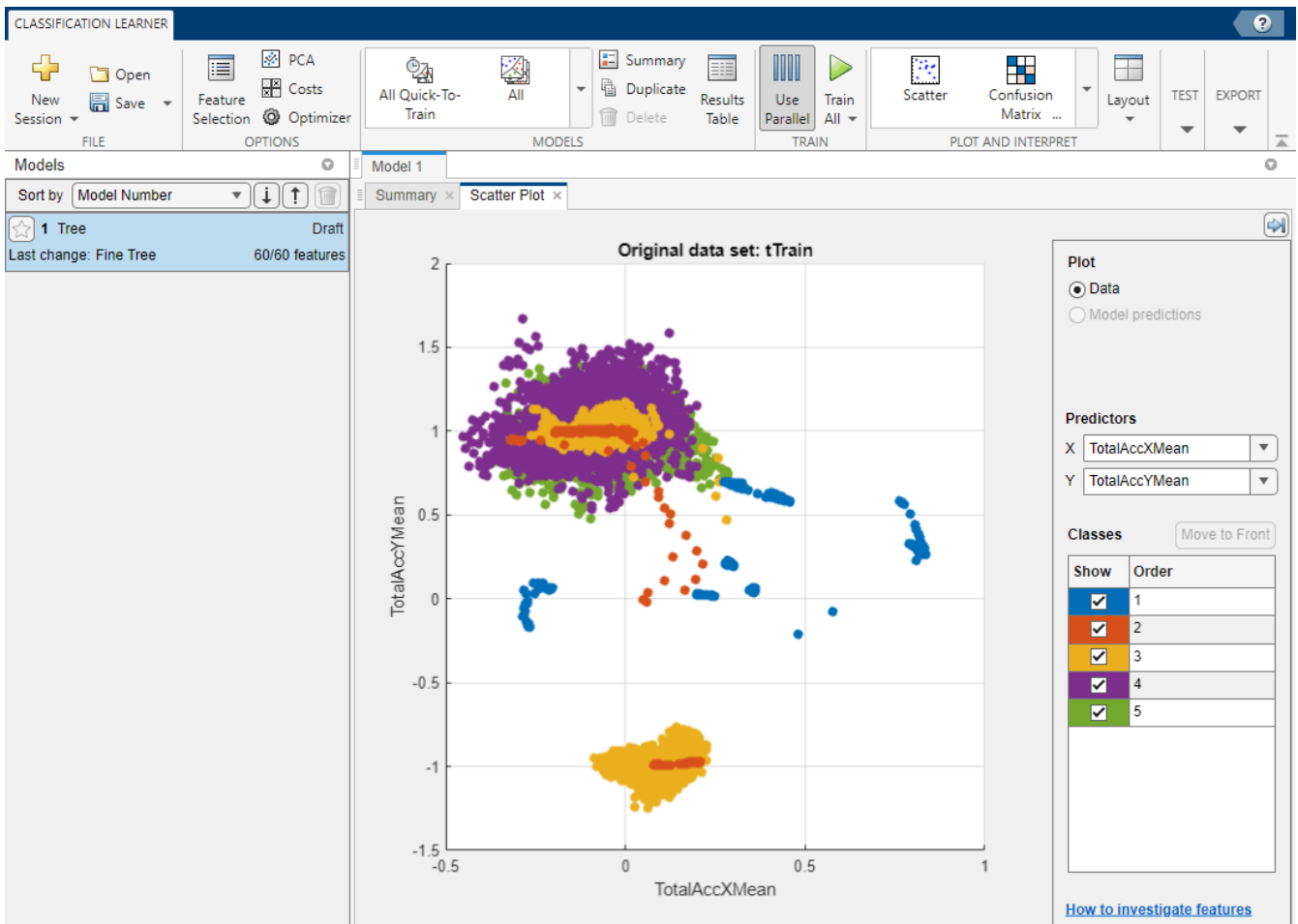
Percent set aside: 10

Use a test set to evaluate model performance after tuning and training models. To import a separate test set instead of partitioning the current data set, use the Test Data button after starting an app session.

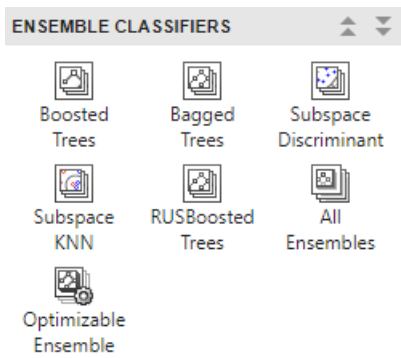
[Read about test data](#)

Start Session Cancel

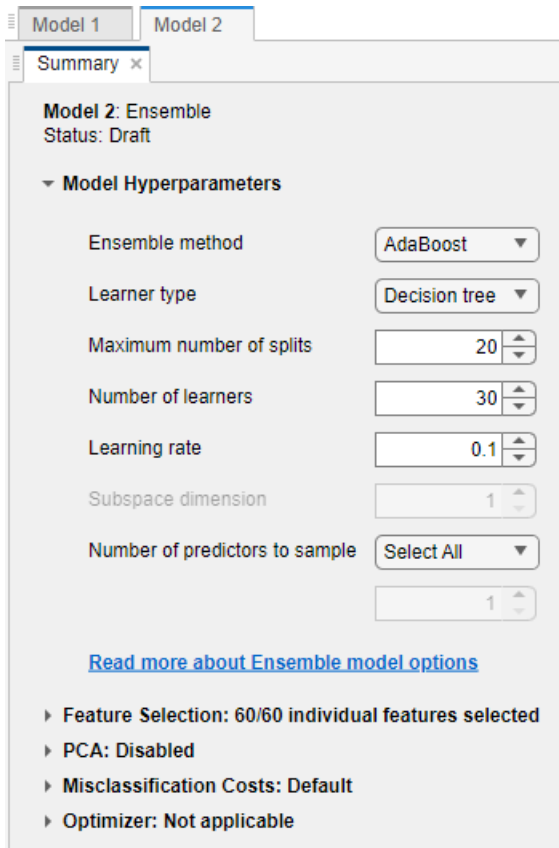
The default option is 5-fold cross-validation, which protects against overfitting. Click **Start Session**. Classification Learner loads the data set and plots a scatter plot of the first two features.



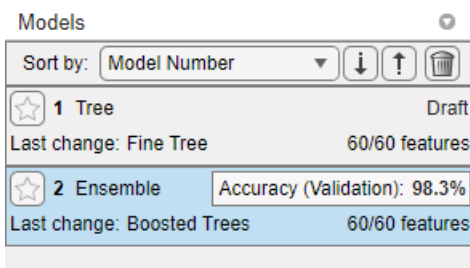
On the **Classification Learner** tab, click the arrow at the right of the **Models** section to open the gallery. Then, under **Ensemble Classifiers**, click **Boosted Trees**.



The model **Summary** tab displays the default settings of the boosted tree ensemble model.



On the **Classification Learner** tab, in the **Train** section, click **Train All** and select **Train Selected**. When the training is complete, the **Models** pane displays the 5-fold, cross-validated classification accuracy.



On the **Classification Learner** tab, click **Export**, click **Export Model**, and select **Export Model**. In the Export Classification Model dialog box, clear the check box to exclude the training data and export a compact model, and then click **OK**. The structure `trainedModel` appears in the MATLAB® workspace. The field `ClassificationEnsemble` of `trainedModel` contains the compact model. Extract the trained model from the structure.

```
classificationEnsemble = trainedModel.ClassificationEnsemble;
```

Train Boosted Tree Ensemble at Command Line

Alternatively, you can train the same classification model at the command line.

```
template = templateTree('MaxNumSplits',20,'Reproducible',true);
classificationEnsemble = fitcensemble(XTrain,YTrain, ...
    'Method','AdaBoostM2', ...
    'NumLearningCycles',30, ...
    'Learners',template, ...
    'LearnRate',0.1, ...
    'ClassNames',[1; 2; 3; 4; 5]);
```

Get ▾

Perform 5-fold cross-validation for `classificationEnsemble` and compute the validation accuracy.

```
partitionedModel = crossval(classificationEnsemble,'KFold',5);
validationAccuracy = 1-kfoldLoss(partitionedModel)
```

Get ▾

```
validationAccuracy = 0.9833
```

Evaluate Performance on Test Data

Evaluate performance on the test data set.

```
testAccuracy = 1-loss(classificationEnsemble,XTest,YTest)
```

Get ▾

```
testAccuracy = 0.9759
```

The trained model correctly classifies 97.59% of the human activities on the test data set. This result confirms that the trained model does not overfit to the training data set.

Note that the accuracy values can vary slightly depending on your operating system.

Save Trained Model

For code generation including a classification model object, use `saveLearnerForCoder` and `loadLearnerForCoder`.

Save the trained model by using `saveLearnerForCoder`.

```
saveLearnerForCoder(classificationEnsemble,'EnsembleModel.mat');
```

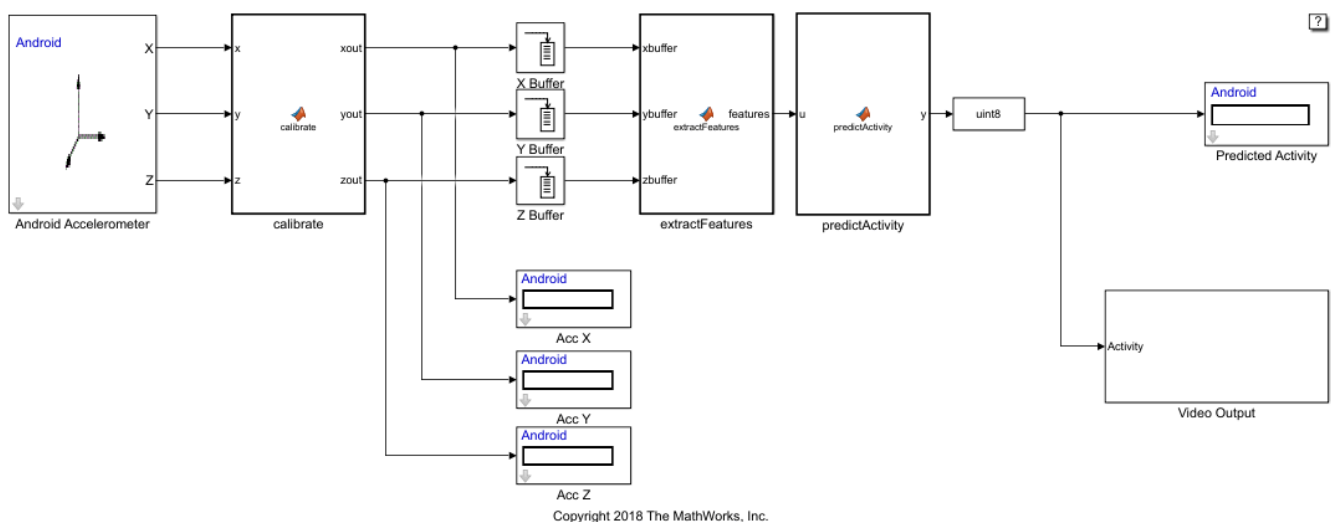
Get ▾

The function block **predictActivity** in the Simulink model loads the trained model by using `loadLearnerForCoder` and uses the trained model to classify new data.

Deploy Simulink Model to Device

Now that you have prepared a classification model, you can open the Simulink model and deploy the model to your device. Note that the Simulink model requires the `EnsembleModel.mat` file and the calibration matrix file `slexHARAndroidCalibrationMatrix.mat`. If you click the button located in the upper-right section of this page and open this example in MATLAB, then MATLAB opens the example folder that includes these calibration matrix files.

Type `slexHARAndroidExample` to open the Simulink model for Android deployment.

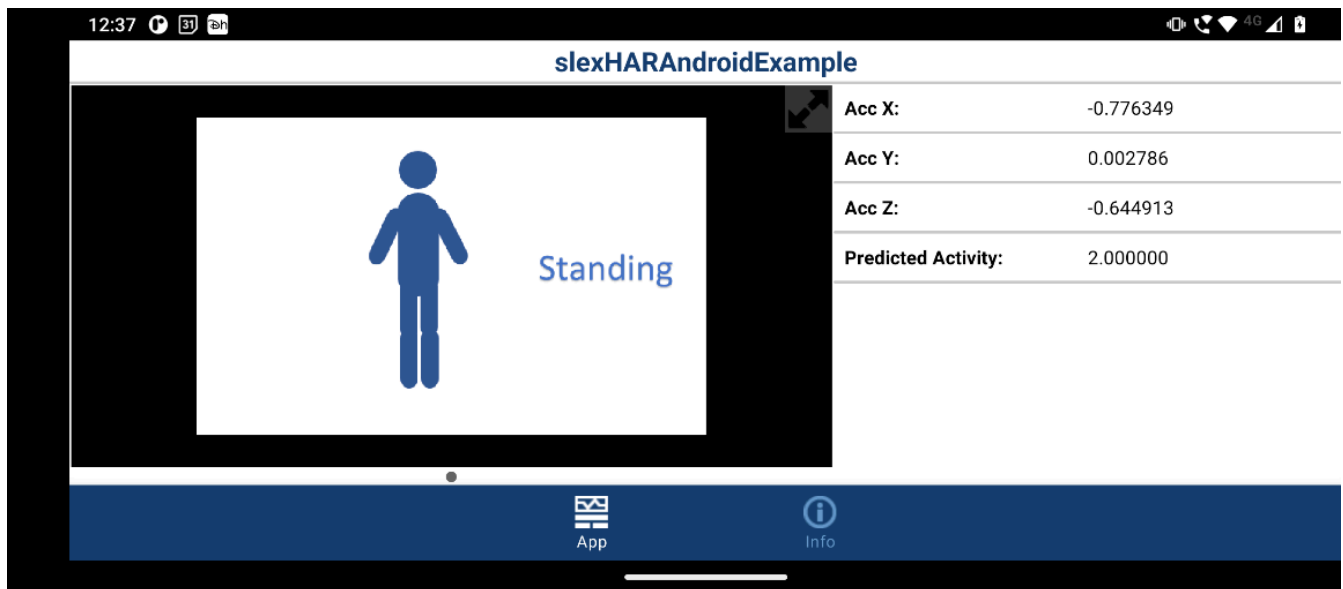


The Simulink model classifies human activity based on acceleration data measured by a smartphone sensor. The model includes the following blocks:

- The **Accelerometer** block receives raw acceleration data from accelerometer sensors on the device.
- The **calibrate** block is a MATLAB Function block that calibrates the raw acceleration data. This block uses the calibration matrix in the `slexHARAndroidCalibrationMatrix.mat` file. If you click the button located in the upper-right section of this page and open this example in MATLAB, then MATLAB opens the example folder that includes this file.

- The display blocks **Acc X**, **Acc Y**, and **Acc Z** are connected to the **calibrate** block and display calibrated data points for each axis on the device.
- Each of the **Buffer** blocks, **X Buffer**, **Y Buffer**, and **Z Buffer**, buffers 32 samples of an accelerometer axis with 12 samples of overlap between buffered frames. After collecting 20 samples, each **Buffer** block joins the 20 samples with 12 samples from the previous frame and passes the total 32 samples to the **extractFeatures** block. Each **Buffer** block receives an input sample every 0.1 second and outputs a buffered frame including 32 samples every 2 seconds.
- The **extractFeatures** block is a MATLAB Function block that extracts 60 features from a buffered frame of 32 accelerometer samples. This function block uses DSP System Toolbox™ and Signal Processing Toolbox™.
- The **predictActivity** block is a MATLAB Function block that loads the trained model from the EnsembleModel.mat file by using loadLearnerForCoder and classifies the user activity using the extracted features. The output is an integer between 1 and 5, corresponding to Sitting, Standing, Walking, Running, and Dancing, respectively.
- The **Predicted Activity** block displays the classified user activity values on the device.
- The **Video Output** subsystem uses a multiport switch block to choose the corresponding user activity image data to display on the device. The **Convert to RGB** block decomposes the selected image into separate RGB vectors and passes the image to the **Activity Display** block.

To deploy the Simulink model to your device, follow the steps in Run Model on Android Devices (Simulink). Run the model on your device, place the device in the same way as described earlier for collecting the training data, and try the five activities. The model displays the classified activity accordingly.



To ensure the accuracy of the model, you need to place your device in the same way as described for collecting the training data. If you want to place your device in a different location or orientation, then collect the data in your own way and use your data to train the classification model.

The accuracy of the model can be different from the accuracy of the test data set (testaccuracy), depending on the device. To improve the model, you can consider using additional sensors and updating the calibration matrix. Also, you can add another output block for audio feedback to the output subsystem using Audio Toolbox™. Use a ThingSpeak™ write block to publish classified activities and acceleration data from your device to the Internet of Things. For details, see <https://thingspeak.com/>.

References

- [1] El Helou, A. Sensor HAR recognition App. MathWorks File Exchange <https://www.mathworks.com/matlabcentral/fileexchange/54138-sensor-har-recognition-app>
- [2] STMicroelectronics, AN4508 Application note. "Parameters and calibration of a low-g 3-axis accelerometer." 2014.
- [3] El Helou, A. Sensor Data Analytics. MathWorks File Exchange <https://www.mathworks.com/matlabcentral/fileexchange/54139-sensor-data-analytics-french-webinar-code>

See Also

loadLearnerForCoder | saveLearnerForCoder | fitcensemble | predict

Related Topics

- Introduction to Code Generation
- Code Generation for Image Classification
- Predict Class Labels Using MATLAB Function Block
- System Objects for Classification and Code Generation
- Predict Class Labels Using Stateflow